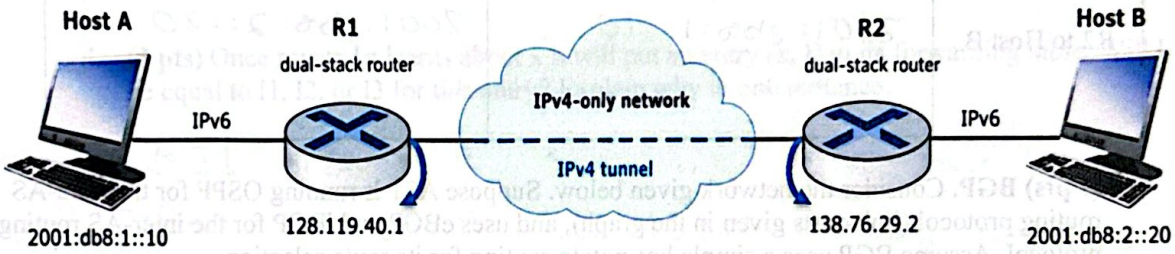
Name: Mitul Marimuthu *(as it would appear on official course roster)*	
UCSB email address: mmarimuthu	**@ucsb.edu**

Homework 04: IPv6, Generalized Forwarding, Routing Algorithms, BGP
Points: *100*

- MAY ONLY BE TURNED IN ON **GRADESCOPE as a PDF file.**
- There is NO MAKEUP for missed assignments.
- We are strict about enforcing the LATE POLICY for all assignments (see syllabus).
  Only use the space provided for answers. Use clear and clean handwriting (or typing).
  NOT USING THIS TEMPLATE WILL LOSE YOU 20 POINTS!

---

1. **(8 pts) IPv6 Tunneling.** Consider the network shown below. Host A and Host B are IPv6 hosts. Routers R1 and R2 are dual-stack routers, meaning they support both IPv6 and IPv4. The network between R1 and R2 is IPv4-only, so IPv6 packets must be tunneled through the IPv4 network.



Host A sends an IPv6 datagram to Host B.

Addresses:

- Host A IPv6 address: **2001:db8:1::10**
- Host B IPv6 address: **2001:db8:2::20**
- R1 IPv4 address on the IPv4 network: **128.119.40.1**
- R2 IPv4 address on the IPv4 network: **138.76.29.2**

Assume R1 is the tunnel entry point and R2 is the tunnel exit point.

a. **(2 pts)** Before the packet reaches R1, what are the source and destination IP addresses in the original datagram?

Location	Source address	Destination address
Host A to R1	2001:db8:1::10	2001:db8:2::20

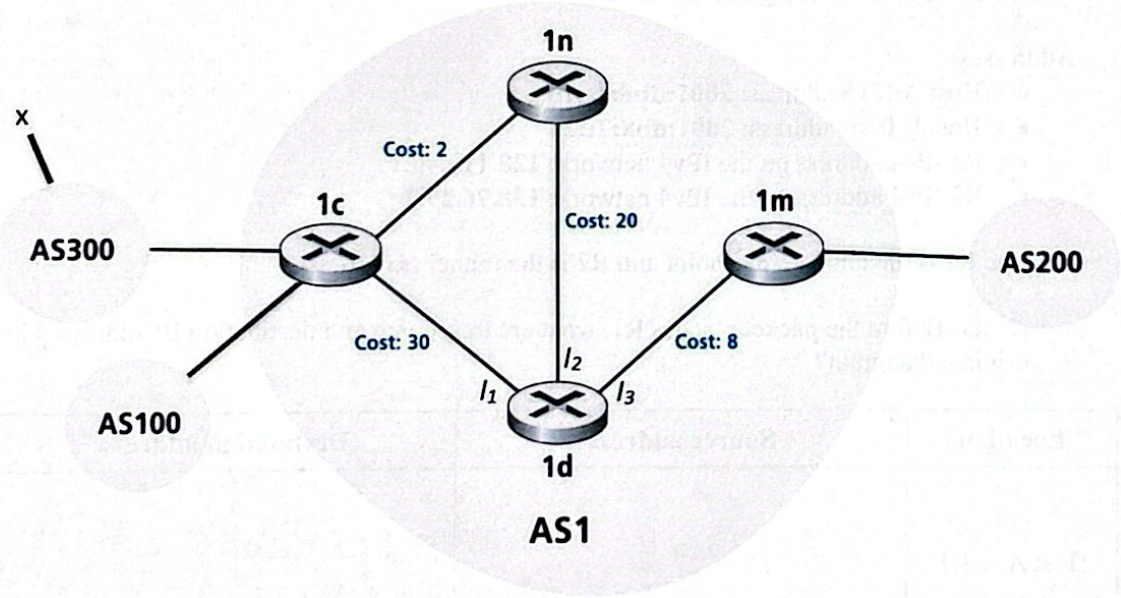
b. **(4 pts)** While the packet is traveling through the IPv4 tunnel from R1 to R2, fill in both the outer datagram header and the inner datagram header.

Location	Outer Datagram Source	Outer Datagram Destination	Inner Datagram Source	Inner Datagram Destination
R1 to R2 through IPv4 tunnel	128.119.40.1	136.76.29.2	2001:db8:1::10	2001:db8:2::20

c. **(2 pts)** After the packet exits the tunnel at R2 and is forwarded to Host B, what are the source and destination IP addresses in the datagram?

Location	Source address	Destination address
R2 to Host B	2001:db8:1::10	2001:db8:2::20

2. **(9 pts) BGP.** Consider the network given below. Suppose AS1 is running OSPF for the intra-AS routing protocol (link costs given in the graph), and uses eBGP and iBGP for the inter-AS routing protocol. Assume BGP uses a simple hot-potato routing for its route selection.



a. **(1 pt)** Which routing protocol (OSPF, eBGP, or iBGP) does router 1c use to learn about prefix x?

eBGP → used between routers in different autonomous systems

b. **(1 pt)** Which routing protocol (OSPF, eBGP, or iBGP) does router 1m use to learn about prefix x?

iBGP. Propagated from 1c after it learns about x

c. **(1 pt)** Which routing protocol (OSPF, eBGP, or iBGP) does router 1c use to learn about router 1n?

OSPF → Both inside AS1

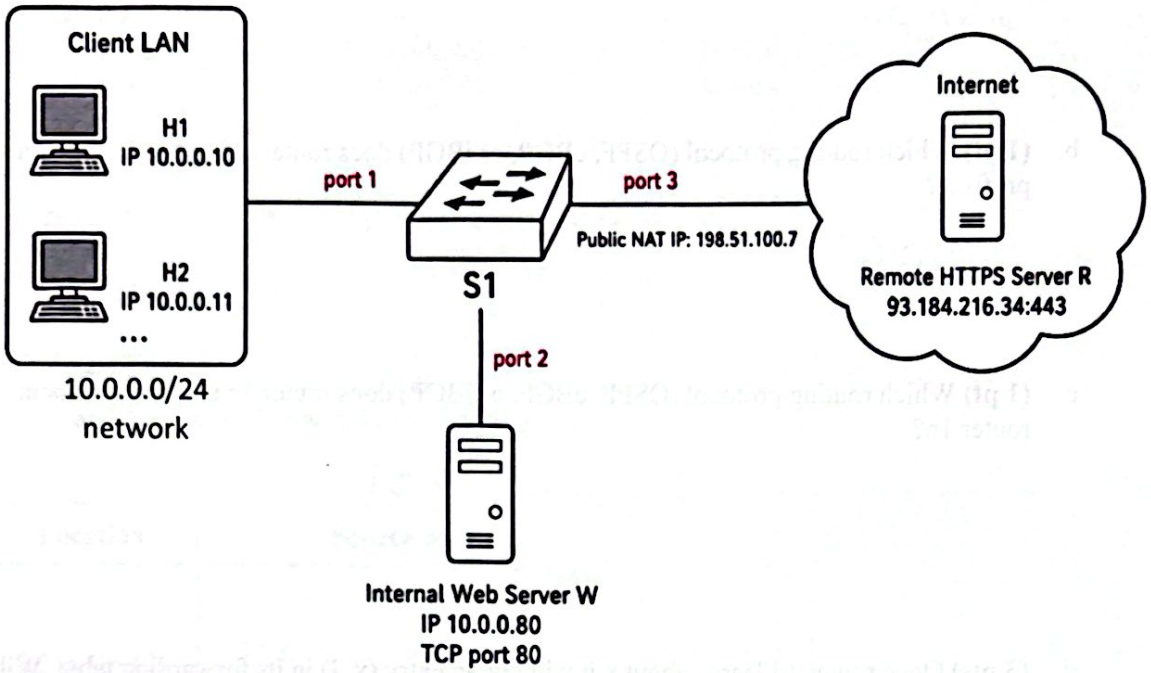
d. **(3 pts)** Once router 1d learns about x it will put an entry (x, I) in its forwarding table. Will I be equal to I1, I2, or I3 for this entry? Explain why in one sentence.

I = I1 because 1c is the only gateway that learned about prefix x via eBGP from AS300, so 1d forwards out interface I1 toward 1c.

e. **(3 pts)** Now suppose router 1d learns that a different network y is accessible via **AS200 AS201 AS237** as well as via **AS100**. It adds an entry (y, J) in its forwarding table. Will J be set to I1, I2, or I3? Explain why in one sentence.

J = I3. Network y is accessible via 1m and 1c, so hot-potato routing picks the gateway with the smallest intra-AS cost which is toward 1m via interface I3.

3. **(21 pts) OpenFlow Generalized Forwarding.** Consider the following OpenFlow network. Switch S1 is an OpenFlow switch at the edge of a small campus network.



Assume S1 supports OpenFlow matching on:

- `in_port`: physical port on the switch (e.g., port 1, 2, or 3) where packet was physically received
- `ip_proto`: specifies which protocol is encapsulated in the payload of IP datagram, such as tcp, udp, etc.
- `ipv4_src`: 32-bit source IPv4 address of the sender
- `ipv4_dst`: 32-bit destination IPv4 address of the sender
- `tcp_src`, `tcp_dst`: source and destination port numbers used by TCP
- `udp_src`, `udp_dst`: source and destination port numbers used by UDP

and supports the following actions:

- output:<port> → forward the datagram to specified port
- drop → drop the datagram
- set_field:<field>=<value> → update field in header to the specified value

a. **(6 pts) Firewall Rules.** Assume that S1 follows a **strict policy**: any traffic that does not match one of the rules below is **dropped**. Write OpenFlow match + action rules for the following permitted traffic:
   - **Rule 1:** Allow the **Internal Web Server W** to send any **TCP** traffic to the **Internet**.
   - **Rule 2:** Host **H1** is allowed to send any traffic to the **Internet** via Port 3.
   - **Rule 3:** Host **H2** is allowed to access the **Internal Web Server W** using **TCP port 80**.
   - **Rule 4:** Allow the **entire Client LAN** to send **DNS queries** to the Internet (Port 3) using **UDP port 53**.

Rule	Match Fields	Action
1	in_port = 2, ipv4_src = 10.0.0.80, ip_proto = tcp	output:3
2	in-port = 1, ipv4-src = 10.0.0.10	output = 3
3	in-port = 1, ipv4-src = 10.0.0.11, ipv4-dst: 10.0.0.80, ip-proto = tcp, tcp-dst = 80	output: 2
4	in-port = 1, ipv4-src = 10.0.0.0/24, ip-proto = udp, udp-dst = 53	output = 3

b. **(9 pts) NAT Using OpenFlow Header Rewriting.** The switch S1 also performs NAT for two existing TCP connections. NAT table below:

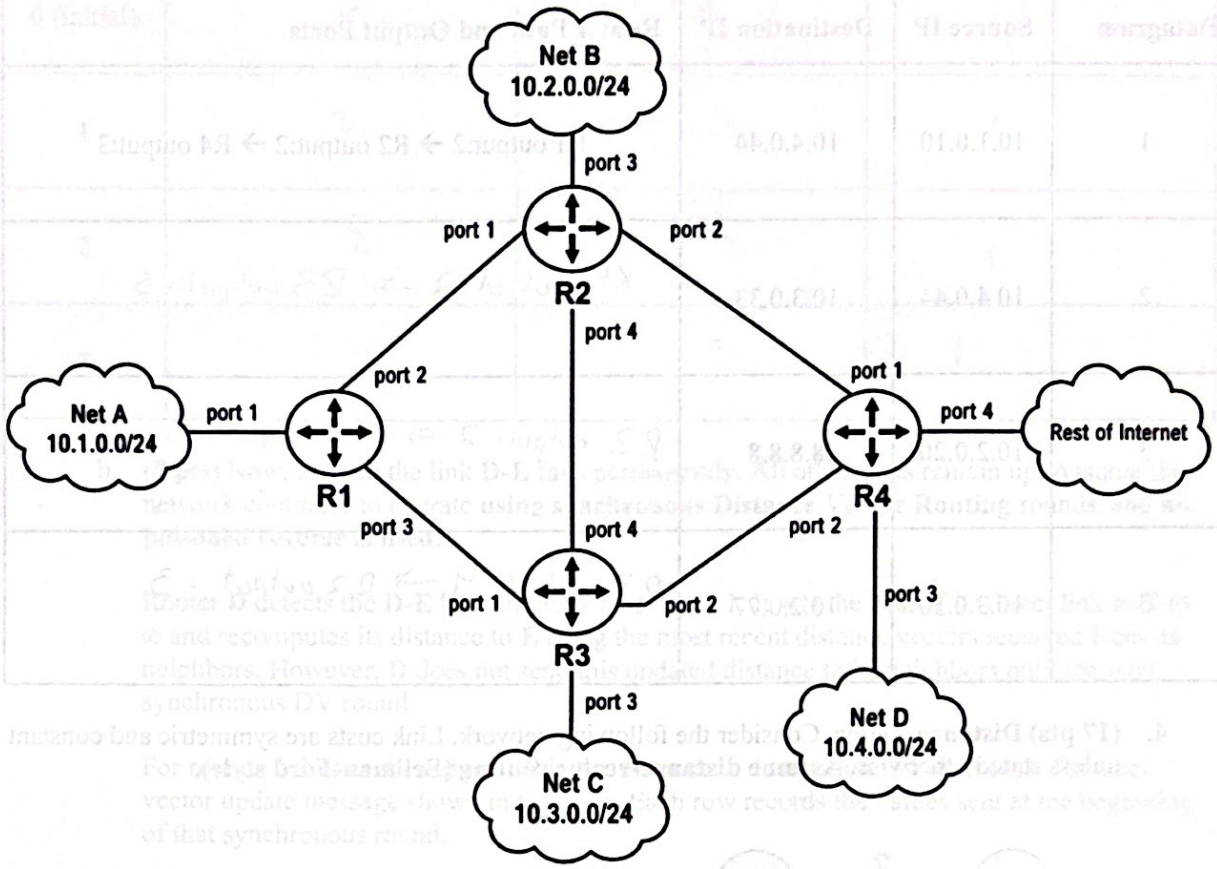
WAN Side	LAN Side	Remote Server
198.51.100.7:40001	10.0.0.10:51515	93.184.216.34:443
198.51.100.7:40002	10.0.0.11:51515	93.184.216.34:443

Note: While we used a simplified NAT table in lecture, modern routers identify specific connections by matching on all four primary fields: ***Source IP, Source Port, Destination IP, and Destination Port****. This allows the router to track the remote server details for every individual connection.*

For each row below, write the OpenFlow match fields and actions needed to implement the NAT translation.

Flow Direction	Match Fields	Actions
H1 → Internet	in_port=1, ip_proto=tcp, ipv4_src=10.0.0.10, tcp_src=51515, ipv4_dst=93.184.216.34, tcp_dst=443	set_field:ipv4_src=198.51.100.7, set_field:tcp_src=40001, output:3
Internet → H1	in-port=3, ip-proto=tcp, ipv4-src=93.184.216.34, tcp-src=443, ipv4-dst=198.51.100.7, tcp-dst=40001	set-field:ipv4-dst=10.0.0.10, set-field:tcp-dst=51515, output=1
H2 → Internet	in-port=1, ip-proto=tcp, ipv4-src=10.0.0.11, tcp-src=51515, ipv4-dst=93.184.216.34, tcp-dst=443	set-field:ipv4-src=198.51.100.7 set-field:tcp-src=40002. output=3
Internet → H2	in-port=3, ip-proto=tcp ipv4-src=93.184.216.34, tcp-src=443, ipv4-dst=198.51.100.7, tcp-dst=40002	set-field:ipv4-dst=10.0.0.11 set-field:tcp-dst=51515 output=1

c. **(6 pts) Destination-Based Forwarding with OpenFlow Tables.** Now consider a different network of four OpenFlow switches acting as routers.



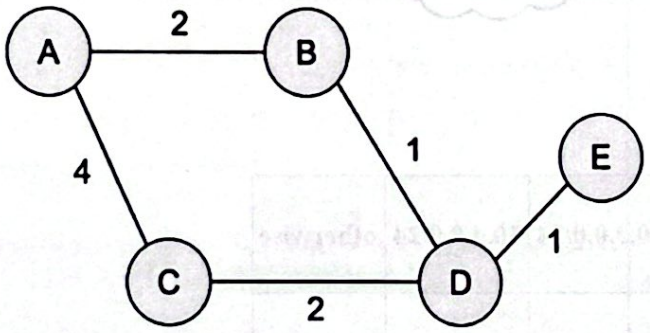
OpenFlow forwarding tables:

Router	10.1.0.0/24	10.2.0.0/24	10.3.0.0/24	10.4.0.0/24	otherwise
R1	output:1	output:2	output:3	output:2	output:2
R2	output:1	output:3	output:4	output:2	output:2
R3	output:1	output:4	output:3	output:2	output:2
R4	output:1	output:1	output:2	output:3	output:4

For each datagram below, list the router-by-router path taken by the packet. Include the final output port used at each router.

Datagram	Source IP	Destination IP	Router Path and Output Ports
1	10.1.0.10	10.4.0.44	R1 output:2 → R2 output:2 → R4 output:3
2	10.4.0.44	10.3.0.33	R4 output: 2 → R3 output: 3
3	10.2.0.20	8.8.8.8	R2 output: 2 → R4 output: 4
4	10.3.0.30	10.2.0.77	R3 output: 4 → R2 output = 3

4. **(17 pts) Distance vector.** Consider the following network. Link costs are symmetric and constant unless stated otherwise. **Assume distance-vector routing (Bellman-Ford style).**



Initially, each router knows only the cost to itself and to its directly connected neighbors. All unknown destinations are initialized to ∞.

a. **(6 pts)** Fill in the following table with the distance to destination **E** computed by each router at initialization and after each synchronous distance-vector round when all links are working normally.

Round	$D_B(E)$	$D_C(E)$	$D_D(E)$
0 (initial)	∞	∞	1
1	2	3	1
2	2	3	1
3	2	3	1

b. **(8 pts)** Now, assume the link **D-E** fails permanently. All other links remain up. Assume the network continues to operate **using synchronous Distance Vector Routing** rounds, and **no poisoned reverse is used**.

Router **D** detects the **D-E** link failure before round 1. It sets the cost of its direct link to **E** to ∞ and recomputes its distance to **E** using the most recent distance vectors received from its neighbors. However, **D** does not send this updated distance to its neighbors until the *next* synchronous DV round.

For rounds 1-3 after the failure, fill in the distance to destination E sent in each distance-vector update message shown in the table. Each row records the values sent at the beginning of that synchronous round.

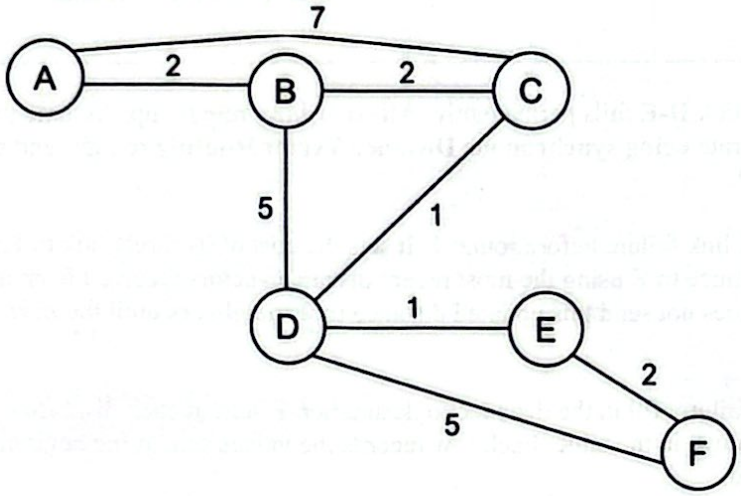
Round	$C \rightarrow D$	$D \rightarrow C$
0 (before D-E link failure)	3	1
1	3	5
2	7	5
3	7	9

*Note: Each entry is the advertised distance to destination **E** in that DV message.*

c. **(3 pts)** Now suppose poisoned reverse is used. Immediately after the D-E link failure, what distance to destination E does router C advertise to router D? Briefly explain why.

C advertises ∞ to D. With poisoned reverse, if C's best route to E is through D, C will advertise ∞ to D to prevent D from thinking it can reach E through C, breaking the routing loop before it starts. This avoids the count-to-infinity problem entirely.

5. **(22 pts) Link State Routing.** Consider the following undirected network graph. Each edge is labeled with its link cost.



a. **(3 pts)** Complete the link-state database after all link-state advertisements have been flooded successfully.

Router	Directly Connected Neighbors and Link Costs
A	B:2, C:7
B	A: 2, C: 2, D: 5
C	A: 7, B: 2, D: 1
D	B: 5, C: 1, E: 1, F: 5
E	D: 1, F: 2
F	D: 5, E: 2

b. **(8 pts)** Run Dijkstra's algorithm from router A. Complete the table below. Use notation D(X), p(X) where D(X) is the current least-cost estimate from A to X, and p(X) is the predecessor of X on that path.

Step	Confirmed Set N′	D(B), p(B)	D(C), p(C)	D(D), p(D)	D(E), p(E)	D(F), p(F)
0	{A}	2, A	7, A	∞, -	∞, -	∞, -
1	{A, B}	2, A	4, B	7, B	∞, -	∞, -
2	{A, B, C}	2, A	4, B	5, C	∞, -	∞, -
3	{A, B, C, D}	2, A	4, B	5, C	6, D	10, D
4	{A, B, C, D, E}	2, A	4, B	5, C	6, D	8, E
5	{A, B, C, D, E, F}	2, A	4, B	5, C	6, D	8, E

c. **(5 pts)** Using the shortest paths found in part (b), complete router A's forwarding table. Assume each router has a directly attached subnet:

Router	Attached Subnet
A	10.10.0.0/24
B	10.20.0.0/24
C	10.30.0.0/24
D	10.40.0.0/24
E	10.50.0.0/24
F	10.60.0.0/24

Destination Subnet	Next Hop from A	Total Path Cost
10.20.0.0/24	B	2
10.30.0.0/24	B	4
10.40.0.0/24	B	7
10.50.0.0/24	B	6
10.60.0.0/24	B	8

d. **(6 pts)** Assume the following simplified flooding model: each router originates one link-state packet, and each link-state packet is eventually sent once over every directed link in the network. Do not count acknowledgments, retransmissions, or periodic refresh messages.

i. **(1 pt)** How many undirected links are in the graph?

8

ii. **(1 pt)** How many directed link transmissions exist in the graph?

16 directed links
+ 6 routers (origin)
96 direct link transmissions

iii. **(4 pts)** How many total link-state packet transmissions are needed for one complete flooding round in which every router's link-state packet reaches every other router? Show your work.

96.

8 undirected links
× 2
16 directed links
× 6 origin routers
96 link-state packet transmissions

6. **(8 pts) True or False.** Indicate whether each statement is **True (T)** or **False (F)**.

a. In SDN, the control plane is logically centralized in the controller, while switches mainly perform data-plane forwarding.

True.

b. Traceroute works by sending packets with the same TTL value each time and observing ICMP messages returned by routers.

False. Send incrementally increasing TTL values

c. The link layer is implemented only in end hosts, not in routers or switches.

False. implemented in all network devices

d. The count-to-infinity problem is mainly associated with link-state routing, not distance-vector routing.

False. associated with distance-vector routing.

e. IPv6 routers perform fragmentation in the same way that IPv4 routers do.

Fake. If packet is too large it is dropped mid-path

f. The common DHCP address-allocation sequence is Discover, Request, Offer, Acknowledgment.

fake. Discover, Offer, Request, Acknowledgement.

g. If an organization's network is part of the global Internet, it must use the same routing protocol for internal traffic as it does for exchanging routes with other ISPs to ensure compatibility.

Fake. Free to use different protocols for intra and inter

h. Under "Hot Potato Routing," a router in an AS will choose a gateway to an external destination based on the shortest path within its own network, even if that gateway leads to a longer path across the rest of the Internet.

True, Hot Potato routing wants to get traffic out of it's own AS as quickly as possible.

(15 pts) Wireshark Lab Handin

Please follow the Wireshark lab writeup and provide your answers to the following questions.

1. **(2 pts)** What is the **IP address of the client** that sends the HTTP GET request in the *NAT_home_side.pcap* trace? What is the **source port number** of the TCP segment in this datagram containing the HTTP GET request? What is the **destination IP address** of this HTTP GET request? What is the **destination port number** of the TCP segment in this datagram containing the HTTP GET request?

src IP: 192.168.10.11

src port: 53924

dst IP: 128.76.29.8

dst port: 80

2. **(1 pt)** At what time[1] is the corresponding HTTP 200 OK message from the webserver forwarded by the NAT router to the client on the router's LAN side?

20:50:27.774683377

[1] Specify time using the time since the beginning of the trace (rather than absolute, wall-clock time).

3. **(2 pts)** What are the source and destination IP addresses and TCP source and destination ports on the IP datagram carrying this HTTP 200 OK message?

src IP: 128.76.29.8
src port: 80
dst IP: 192.168.10.11
dst port: 53124

4. **(1 pt)** At what time does this HTTP GET message appear in the *NAT_ISP_side.pcap* trace file?

20:50:27.771391145

5. **(2 pts)** What are the source and destination IP addresses and TCP source and destination port numbers on the IP datagram carrying this HTTP GET (as recorded in the *NAT_ISP_side.pcap* trace file)?

source IP: 10.0.1.254
src port: 53924
dst IP: 128.76.29.8
dst port: 80

6. **(1 pt)** Which of these four fields are different than in your answer to question 1 above?

only source IP has changed

7. **(1 pt)** Are any fields in the HTTP GET message changed?

No, NAT only modifies the IP/TCP headers, not the payload.

8. **(1 pt)** Which of the following fields in the IP datagram carrying the HTTP GET are changed from the datagram received on the local area network (inside) to the corresponding datagram forwarded on the Internet side (outside) of the NAT router: Version, Header Length, Flags, Checksum?

Checksum only => re computed after source IP is rewritten.

9. **(1 pt)** At what time does this message appear in the *NAT_ISP_side.pcap* trace file?

20:50:27.774660620

10. **(2 pts)** What are the source and destination IP addresses and TCP source and destination port numbers on the IP datagram carrying this HTTP reply ("200 OK") message (as recorded in the *NAT_ISP_side.pcap* trace file)?

src IP: 138.76.29.8
src port: 80
dst IP: 10.0.1.254
dst port: 53924

11. **(1 pt)** What are the source and destination IP addresses and TCP source and destination port numbers on the IP datagram carrying the HTTP reply ("200 OK") that is forwarded from the router to the destination host in the right of Figure 1?

src IP: 138.76.29.8
src port: 80
dst IP: 192.168.10.11
dst port: 53924